

BLE Avançado: Gateway Raspberry, Cliente BLE e Notificações

Tecnologias de Comunicação para a Internet das Coisas:
Sebenta da Aula 06

Catarina Silva

2026

Abstract

Este documento acompanha a sexta aula da unidade curricular, de natureza prática. Explica os conceitos que sustentam os quatro exercícios propostos: a administração remota do Raspberry Pi, a sua configuração como cliente BLE, o funcionamento das notificações assíncronas e a construção de uma ponte entre BLE e Python. Destina-se a ser lido antes da execução dos guiões práticos.

Table of Contents

1 O Raspberry Pi como Gateway

Antes de o Raspberry Pi poder desempenhar o papel de gateway numa solução de IoT, é necessário torná-lo administrável à distância e configurar os serviços de que depende. Este capítulo explica o porquê de cada passo.

1.1 Porquê Administração Remota

Um gateway instalado em ambiente real não tem, em geral, teclado nem monitor. Fica colocado onde a cobertura rádio o exige (num quadro elétrico, numa estufa, num armário técnico) e o acesso físico é inconveniente ou impraticável. Toda a configuração, diagnóstico e atualização passa, por isso, por acesso remoto através da rede.

Esta constatação explica a primeira etapa do trabalho prático. O sistema operativo do Raspberry Pi dispõe de ferramentas de parametrização que permitem seleccionar as interfaces e os serviços ativos, com uma interface gráfica e outra em modo texto. Ambas alteram o comportamento do sistema de forma persistente, mantendo-se as opções escolhidas entre reinícios.

Dois serviços interessam particularmente. O SSH (*Secure Shell*) permite o acesso remoto por linha de comandos, cifrado, e é a via normal de administração de um sistema Linux sem interface gráfica. O VNC permite o acesso remoto com interface gráfica, útil quando se pretende utilizar aplicações que o exijam. Cada instalação de gateway é única e tem requisitos próprios, pelo que a decisão sobre que serviços ativar deve ser tomada caso a caso, ativar serviços desnecessários aumenta a superfície de ataque sem benefício.

1.2 Descoberta do Equipamento na Rede

Para aceder remotamente ao Raspberry Pi é preciso conhecer o seu endereço IP, e esse endereço é normalmente atribuído por DHCP, não sendo conhecido à partida.

O procedimento tem duas etapas conceptuais. Primeiro, identificar a rede em que o computador de administração se encontra, o que se faz consultando a configuração das interfaces de rede locais e observando o endereço IP e a máscara de rede da interface ativa. Daí se deduz a gama de endereços em uso. Segundo, procurar na rede os equipamentos ativos, através de um varrimento

que envia pedidos a todos os endereços dessa gama e regista quais respondem.

Este é um caso em que o guião prático fornece comandos concretos, mas o essencial é o raciocínio: sem conhecer a gama de endereços, o varrimento não tem alvo; sem varrimento, não se localiza um equipamento cujo endereço foi atribuído dinamicamente. Em instalações definitivas, a alternativa é reservar um endereço fixo para o gateway no servidor DHCP da rede, ou configurar um endereço estático no próprio equipamento.

2 Configuração do Raspberry Pi como Cliente BLE

A pilha Bluetooth do Linux não vem, por omissão, configurada para o modo de operação exigido por este exercício. Este capítulo explica que alterações são necessárias e porquê.

2.1 Identificação e Preparação do Sistema

O primeiro passo do guião consiste em alterar o nome do equipamento. A razão é prática: numa sala onde vários grupos trabalham em simultâneo, com equipamentos idênticos, nomes distintos evitam que um aluno se ligue por engano ao dispositivo do colega e passe uma aula a diagnosticar um problema inexistente.

Segue-se a instalação das ferramentas necessárias. O pacote bluez fornece a pilha protocolar Bluetooth do Linux, já apresentada na aula anterior, e o pacote bluez-hcidump acrescenta a capacidade de capturar e analisar o tráfego HCI trocado entre o host e o controlador, instrumento útil quando algo não funciona e é preciso observar o que está efetivamente a ser transmitido.

2.2 Ativação do Modo Low Energy

Duas alterações de configuração são a fonte mais comum de dificuldades neste exercício.

A primeira consiste em ativar as funcionalidades experimentais do serviço Bluetooth, alterando a linha de arranque do processo `bluetoothd` para incluir a opção correspondente. Algumas funcionalidades BLE, sobretudo as relacionadas com a gestão de atributos e notificações, estiveram durante anos marcadas como experimentais no BlueZ, não sendo expostas por omissão. Sem esta alteração, certas operações simplesmente não estão disponíveis, e o erro devolvido não é necessariamente esclarecedor.

A segunda consiste em alterar o modo do controlador para *low energy*, no ficheiro de configuração principal do Bluetooth. Um adaptador de modo duplo suporta simultaneamente Bluetooth clássico e BLE. Fixar o modo em *le* instrui o controlador a operar exclusivamente em Bluetooth Low Energy, o que evita ambiguidades na descoberta de dispositivos e no estabelecimento de ligações. Recorde-se, da aula anterior, que as duas tecnologias não são interoperáveis: um dispositivo à procura de periféricos BLE não encontra dispositivos Bluetooth clássicos, e a confusão entre os dois modos gera resultados difíceis de interpretar.

Ambas as alterações exigem o reinício do serviço para produzirem efeito.

2.3 Descoberta e Exploração do Periférico

Configurado o sistema, o procedimento segue a sequência já introduzida na aula anterior, agora executada no Raspberry Pi.

Começa-se por identificar as interfaces BLE presentes na máquina, o que confirma que o adaptador é reconhecido pelo sistema e revela o seu endereço físico. Passa-se depois ao varrimento de dispositivos Low Energy na

vizinhança. O resultado deste varrimento é instrutivo por si só: numa sala comum, aparecem dezenas de dispositivos, televisores, pulseiras de atividade, auscultadores, telemóveis, muitos deles sem nome anunciado. Relacione-se esta observação com o caso de estudo das pulseiras de atividade discutido na aula anterior: cada uma destas linhas corresponde a um dispositivo que está a difundir a sua presença a quem estiver a escutar.

Identificado o endereço do periférico pretendido, estabelece-se a ligação com o gatttool em modo interativo e emite-se o comando de ligação. Estabelecida a ligação, o comando primary lista os serviços primários expostos pelo dispositivo, apresentando para cada um o intervalo de *handles* que ocupa e o respetivo UUID.

Este resultado é a materialização direta da hierarquia GATT estudada na aula anterior. Os UUIDs que começam por 00001800 e 00001801 correspondem a serviços normalizados pelo Bluetooth SIG, respetivamente o serviço de acesso genérico e o serviço de atributos genéricos, presentes em praticamente todos os dispositivos. Os restantes correspondem a serviços específicos do dispositivo em causa. A partir da lista de serviços, descobrem-se as características de cada um e, dessas, leem-se ou escrevem-se os valores.

3 Notificações Assíncronas e Ponte BLE-Python

A última parte do trabalho prático substitui a interrogação periódica por um modelo orientado a eventos, e liga o resultado a um programa Python. É este passo que prepara a integração com um middleware de mensagens.

3.1 Da Interrogação Periódica às Notificações

Nos exercícios anteriores, o cliente pergunta e o periférico responde. O modelo é simples, mas tem duas fragilidades. A primeira é o consumo: o cliente tem de interrogar repetidamente, mesmo quando nada mudou, e cada interrogação obriga ambos os dispositivos a acordar a rádio. A segunda é a latência: uma alteração no valor de um sensor só é detetada na leitura seguinte, pelo que o atraso médio é metade do período de interrogação.

As notificações resolvem ambos os problemas invertendo a iniciativa. O cliente declara uma vez o seu interesse numa característica, escrevendo no respetivo descritor de configuração, e a partir daí é o servidor que envia dados quando o valor se altera. Entre as duas variantes disponíveis no protocolo, a notificação não exige confirmação do cliente, ao passo que a indicação exige, sendo por isso mais fiável e simultaneamente mais dispendiosa.

O programa Arduino usado nesta parte do trabalho difere do da aula anterior precisamente neste ponto: em vez de responder apenas quando interrogado, envia notificações de acordo com o último pedido recebido do servidor. Do lado do cliente, um programa Python emite pedidos de forma rotativa e imprime os dados recebidos nas notificações.

3.2 A Ponte BLE-Série em Python

O programa cliente assenta na biblioteca bluepy, apresentada na aula anterior, e segue o padrão de delegação característico desta biblioteca.

A estrutura é a seguinte. Define-se uma subclasse de DefaultDelegate que redefine o método `handleNotification`, invocado automaticamente sempre que chega uma notificação; o método recebe o *handle* da característica que a originou e os dados transmitidos, e é aí que se coloca o processamento pretendido. Cria-se depois um objeto `Peripheral` com o endereço do dispositivo (endereço que se obtém com o varrimento descrito no capítulo anterior) e

associa-se-lhe o delegado. Obtém-se o serviço pelo respetivo UUID e, a partir dele, a característica. O programa escreve então na característica para pedir uma grandeza e entra num ciclo de espera por notificações.

Há um detalhe de projeto a reter. Os módulos BLE simples, como o HM-11 usado nestes exercícios, não expõem serviços GATT ricos: expõem uma característica que funciona, na prática, como um canal série sobre BLE. Escrever nessa característica equivale a enviar bytes pela porta série do Arduino; receber uma notificação equivale a ler bytes dessa porta. Daí a designação de “ponte série sobre BLE”, a comunicação é, do ponto de vista da aplicação, um fluxo de caracteres, e não um conjunto estruturado de atributos tipados.

Esta simplificação tem vantagens e custos. Facilita muito a programação, porque dispensa a definição de um perfil GATT próprio. Em contrapartida, perde-se a autodescrição que o GATT oferece: um cliente genérico não consegue descobrir, a partir dos UUIDs, que grandezas o dispositivo disponibiliza nem em que unidades, tem de conhecer previamente a convenção de caracteres usada.

3.3 Onde Este Trabalho Conduz

O comentário deixado no código-fonte original do programa Python (assinalando, no ponto onde os dados da notificação são recebidos, que ali passará a existir um cliente de broker) indica com precisão o destino deste percurso.

Neste momento, o programa Python limita-se a imprimir no ecrã os valores recebidos. O passo seguinte, tratado nas aulas dedicadas aos protocolos de sessão e aos middlewares orientados a mensagens, consiste em substituir essa impressão pela publicação dos valores num broker. É essa substituição que transforma um exercício de laboratório numa arquitetura escalável: a partir do momento em que os dados são publicados num broker, deixam de estar

confinados ao processo que os recolheu e ficam disponíveis para qualquer número de consumidores, mecanismos de análise, alarmística ou serialização e armazenamento.

Ao concluir esta aula, situe-se o trabalho realizado no modelo de referência apresentado na primeira aula. O Arduino com sensores corresponde à camada de dispositivos físicos. A ligação BLE corresponde à camada de conectividade. O Raspberry Pi, ao receber, interpretar e reencaminhar os dados, desempenha funções de computação de borda. As camadas superiores (acumulação, abstração e aplicação) serão construídas na segunda metade do semestre.